

x402 Facilitator & Settlement Mesh — Complete Implementation Plan

Team Agni · Algorand Sponsor Track

0. How Hackathon Judging Actually Works (and how this plan is built to win it)

Sponsor tracks are almost never judged on "did it work." They're judged on five things, roughly in this order of weight:

Axis	What judges are really asking	Where this plan hits it
Sponsor tech depth	Did you use <i>this chain's</i> actual differentiators, or could this be "any EVM chain with the names swapped"?	Atomic transaction groups, Note-field receipts, ASA/USDC support, (stretch) State Proofs — things you <i>cannot</i> do on Ethereum the same way
Technical execution	Does the core loop actually work under adversarial conditions live on stage?	The Chaos Storm demo (§3, Tier 1c) — this is your single highest-leverage 90 seconds
Innovation / novelty	Is there a genuine idea here, not just a CRUD wrapper around a payment?	On-chain audit hash-chain, AI-agent-to-agent payment narrative
Real-world viability	Would this survive contact with a real integrator next week?	Non-custodial guarantee, idempotency correctness, Bazaar discoverability
Presentation	Can a judge who's seen 40 projects today remember yours in one sentence?	"Two AI agents paid each other on Algorand, on-chain, in under 2 seconds, and we broke it live and it didn't break"

Everything below is organized so that a judge watching your demo sees Algorand-specific tech doing something an EVM chain genuinely can't do as cleanly — that's the fastest path to sponsor-track votes.

1. Corrected Core Architecture

1.1 Install & imports (verified against the actual SDK docs)

```
bash
```

```
pip install "x402-avm[fastapi,avm]" redis uvicorn
```

python

```
from x402.http import HTTPFacilitatorClient, FacilitatorConfig
from x402.mechanisms.avm.exact import register_exact_avm_client, register_exact
from x402.mechanisms.avm.signer import FacilitatorAvmSigner
```

Note: the pip package is `x402-avm`, but the import root is `x402` — not `x402_avm`. This is the single most common mistake Claude Code will make scaffolding this if not told explicitly, so it's worth pinning in your CLAUDE.md verbatim.

1.2 Fixed replay-protection logic (atomic claim, not check-then-act)

Your original rule ("check Redis, if exists return 409") has a race window. Two concurrent requests for the same TxID can both pass the check before either writes. Fix: claim atomically.

python

```
# Atomic claim — SET ... NX EX is a single Redis operation, no race window
claimed = redis_client.set(f"txid:{txid}", "PENDING", nx=True, ex=86400)
if not claimed:
    existing = redis_client.get(f"txid:{txid}")
    if existing == b"SETTLED":
        return existing_receipt(txid) # idempotent replay — return prior receipt
    return JSONResponse(status_code=409, content={"error": "duplicate_in_flight"})

# ... verify signature, settle on-chain ...
redis_client.set(f"txid:{txid}", "SETTLED", ex=86400)
```

24h TTL is deliberate — bounds Redis memory growth over the hackathon demo window without breaking any realistic replay-detection window.

1.3 Correct operation order (security-critical)

1. Parse payload
2. Verify AVM signature ← BEFORE touching Redis at all
3. Atomically claim TxID lock ← SET NX EX
4. Settle on Algorand LocalNet
5. Write receipt to Note field
6. Mark Redis key SETTLED

Verifying signature *before* the Redis claim matters: an attacker sending malformed/unsigned payloads shouldn't be able to pollute your idempotency store or probe which TxIDs already exist (info leak).

1.4 Receipt schema (must fit in 1024-byte Note field)

json

```
{"v":1,"txid":"<algo-txid>","status":"settled","amt":"10000","asset":"USDC","ts"
```

Keep the full payload hash truncated to 16 hex chars (8 bytes) — enough to bind the receipt to the exact request without blowing the byte budget. Don't try to cram the whole payment payload in there.

2. Phase-by-Phase Build Plan

Phase 1 — Environment & Tooling

- `algokit init` → FastAPI + Redis scaffold
- `algokit localnet start`, generate 3 test wallets (payer, payee, fee-payer if using fee abstraction)
- Confirm `algokit explore` opens Lora against LocalNet
- **Deliverable:** `GET /health` returns LocalNet round number + Redis ping — proves the whole stack is wired before you write a line of payment logic

Phase 2 — Payload Parser & Signature Validation

- Ingest x402 payment payload (V2 header/body format)
- `register_exact_avm_server` on your `x402ResourceServer`
- Validate against `ExactAvmFacilitatorScheme` — reject anything that doesn't parse or verify, fail closed
- **Deliverable:** malformed payloads reliably 400, valid-but-unclaimed payloads reach Phase 3

Phase 3 — The Vault (Replay Protection)

- Implement the atomic `(SET NX EX)` claim from §1.2
- Unit test: fire the *same* TxID twice concurrently (`asyncio.gather`), assert exactly one settles and one gets 409
- **This test is your Chaos demo's backbone — write it first, reuse it live**

Phase 4 — Chaos Node Simulators

- `/simulator/happy` — clean payload, full settle-and-receipt round trip
- `/simulator/chaos` — fires N duplicate TxIDs concurrently + malformed payloads + artificial timeouts
- Live dashboard (see Tier 1c below) visualizing claims/rejects in real time as chaos runs

Phase 5 — Immutable Receipts

- 0-ALGO self-payment (or atomic group with the real settlement txn) carrying the Note-field receipt from §1.4
- Facilitator response includes the Lora explorer deep link for that TxID
- **Deliverable:** click the link in your dashboard, judge sees the receipt live on-chain

Phase 6 — The Differentiators (below)

3. The "Crazy Stuff" — Tiered by Effort/Impact

Tier 1 — Build these (highest impact per hour, all realistic in a hackathon window)

a. AI-agent-to-agent payment demo. Two lightweight agents: a "buyer" agent (simple loop, could literally call the Claude API) that discovers a 402 response, decides to pay, and retries; a "seller" agent serving a paid resource (e.g., live weather or a mock dataset) behind your facilitator. `(x402-avm)` already ships an `(httpx)` transport that intercepts 402s automatically — this is a few dozen lines, not a research project. This is your narrative hook: *"machines paying machines, no humans in the loop, settled on Algorand in under 2 seconds."* This is the single highest-ROI thing on this list — it's what makes judges stop scrolling their laptop.

b. ASA/USDC support, not just native ALGO. The SDK already supports ASA-denominated pricing (`(price: {asset: USDC_TESTNET_ASA_ID, amount, extra:{decimals}})`). Pricing your paid endpoint in USDC instead of ALGO is a two-line config change but instantly reads as "production-ready payment rail" instead of "hackathon toy currency."

c. Real-time chaos dashboard. A single WebSocket-backed page: live feed of claim attempts (green=settled, red=409, yellow=malformed-rejected) as `/simulator/chaos` fires a duplicate storm. This turns your Phase 4 unit test into a 20-second live visual — genuinely the moment most sponsor-track demos are won or lost, because it's the only part of a payments demo that's *watchable*.

d. On-chain audit hash-chain. Periodically (e.g. every 10 settlements) commit a hash of the current Redis idempotency set to a tiny Algorand app's global state (or Box storage) — a cheap, verifiable "the off-chain vault hasn't been tampered with" anchor. This directly answers the judge question "what stops you from just lying about what's in Redis?" without needing a full smart-contract vault. Query the Kapa MCP server for current Algorand Python / Box-storage syntax before implementing — the contract-language tooling has moved fast and you don't want Claude Code hallucinating deprecated Beaker patterns.

Tier 2 — Stretch goals if Tier 1 finishes early

a. Multi-facilitator mesh with failover. Two facilitator instances behind a thin router; if one is down or slow, requests route to the other, both sharing the Redis vault. Demonstrates the "mesh" half of your own project name, which judges will notice if you *don't* show it.

b. Bazaar discovery extension. `(x402-avm)` ships a discovery extension (`(with_bazaar)`) letting resource servers register as discoverable paid endpoints. Wiring this in means your demo endpoint is technically discoverable by *any* other x402 client on the network — a real, if small, contribution to the ecosystem rather than a closed demo.

Tier 3 — Don't build, but say out loud in the pitch

a. Algorand State Proofs for trustless cross-facilitator verification. State Proofs let a third party verify Algorand ledger state without trusting any single facilitator's API — the natural end-state of a "settlement mesh." Be honest in the plan (and to yourselves): State Proofs are generated on a ~256-round cadence on real networks and don't behave the same way on LocalNet, so this isn't a live-demo feature in a hackathon window. Say explicitly: *"today the mesh trusts facilitator responses; the next step is State-Proof-backed verification so no facilitator needs to be trusted at all."* Naming the real next step, and why you didn't rush it, reads as engineering maturity — a small "we know the limits of what we built" line often lands better with judges than pretending everything is finished.

b. Payment-channel batching via rekey. Batch micro-payments off-chain and settle net amounts periodically using Algorand's rekey feature — a lightning-network-style cost reduction. Same treatment: name it as the roadmap, don't attempt it live.

4. Suggested Repo Structure

```
x402-facilitator/  
├─ src/  
│   ├── main.py           # FastAPI app, health check  
│   ├── vault.py          # Atomic Redis claim logic (§1.2)  
│   ├── parser.py         # Payload validation, signature check  
│   └── settlement.py     # AlgoKit LocalNet settlement + Note-field  
receipt  
│   ├── simulators/  
│   │   ├── happy.py  
│   │   └── chaos.py  
│   └── agents/  
│       ├── buyer_agent.py # Tier 1a  
│       └── seller_agent.py  
├─ dashboard/             # Tier 1c WebSocket live view  
├─ contracts/             # Tier 1d hash-chain anchor app  
├─ tests/  
│   └── test_concurrent_replay.py # The duplicate-TxID race test – write this  
early  
└─ CLAUDE.md
```

5. Two-Minute Demo Script

1. **(15s)** One sentence: "Two AI agents pay each other for API access, settled on Algorand, with cryptographic replay protection — non-custodial, no database of record needed."
2. **(30s)** Buyer agent hits a paid endpoint, gets 402, pays in USDC, retries, gets data. Click through to Lora, show the Note-field receipt live.
3. **(45s)** Fire the chaos storm — duplicate TxID storm hits the dashboard live, judges watch it hold at exactly one settlement, N-1 clean 409s.
4. **(20s)** "Here's the roadmap" slide — State Proofs, payment channels — showing you know exactly where the edges of today's build are.
5. **(10s)** Close on the one-sentence hook again.

6. Risk / Mitigation

Risk	Mitigation
LocalNet flakiness during live demo	Record a backup clip of the chaos storm the night before; run live but have the clip ready
Redis race condition resurfacing under real concurrency	The <code>test_concurrent_replay.py</code> test is your proof — run it on stage if asked "how do you know it's safe?"
Judges unfamiliar with x402	The 15-second opening line above should assume zero context
Scope creep into Tier 2/3 eating Tier 1 time	Build order is Tier 1 → nothing else until all four Tier 1 items work end-to-end

7. Time-Boxing

This assumes a fairly typical ~24-36 hour hackathon window — tell me your actual remaining hours and I'll compress or expand the phase allocation accordingly. Rough split: Phases 1-3 (30%), Phase 4 + Tier 1c dashboard (25%), Tier 1a/b (30%), buffer + pitch prep (15%). Buffer is non-negotiable — it's what Tier 2 eats into if you're ahead of schedule, and what protects the demo if you're behind.